FAIRwDDi WP3–ST3 Phase I: Audit and DDI-L Database Model

V1.0.0

Summary

This report concludes **Phase I (Audit and Modeling)** of the FAIRwDDI WP3-ST3 project and delivers a comprehensive technical audit of the current ReQuest platform, plus a validated, standard-agnostic database architecture aligned with **DDI-Lifecycle 3.3**, **DDI 4.0 (COGS)**, and **DDI-CDI**.

Key Takeaways & Architectural Highlights

- **From Pseudo-DDI to Canonical Variable Cascade:**
  - Decoupled question wording from variable representations by introducing a dedicated `QuestionItem` entity.
  - Formalized the full three-tier variable cascade (`ConceptualVariable` → `RepresentedVariable` → `InstanceVariable`), enabling cross-survey harmonization and longitudinal question tracking.
  - Separated numerical response codes (`CodeList` / `Code`) from semantic classification labels (`Category`).
- **First-Class Multilingual & Extensible Metadata:**
  - Replaced monolingual text fields with PostgreSQL **JSONB faceted string arrays** supporting ISO 639-1 language tags (`{"lang": "fr", "value": "..."}`), normalized comparison strings, and translation provenance.
  - Added self-describing `extended_attributes` JSONB storage to capture rich DDI-L attributes (interviewer instructions, question intent, universes) without SQL schema bloat.
- **Persistent Identification & Integrity:**
  - Standardized on canonical **DDI URNs** (`urn:ddi:fr.cdsp:...`) as primary keys across all resources.
  - Introduced multi-algorithm **content fingerprinting** (`hashes` JSONB) to automate deduplication, detect metadata drift, and power flexible search indexing.
  - Added a central `UrnRegistry` and a lightweight `SemanticRelationship` triple store (SKOS/XKOS/PROV-O) for vocabulary and concept anchoring (e.g., CESSDA ELSST).
- **Pragmatic, Lightweight Design for ReQuest:**
  - Avoided monolithic DDI-L specification bloat by implementing streamlined **shortcut association tables** connecting questions, variables, instruments, and study units.
  - Replaced legacy organizational tables (`Collection` , `Subcollection` , `Distributor`) with standard `Organization` and nestable `Group` entities.
  - Established a two-stage staging and ingestion model (`StagedImport` , `StagedResourceNode` , `MetadataQuarantine`), laying the foundation for Phase II multi-standard ingestion (DDI-L 3.3, DDI-C 2.5, DDI 4).

---

1. Audit

The following findings about the current ReQuest database are based on a Postgres dump/copy we received in July, analysis of the application code, and conversations with the Sciences Po team.

1.1 Database schema

- The current ReQuest database consists of 13 tables
- It partially mimics the variable cascade model but is primarily designed to support the ReQuest application
- Its content is based on a normalized subset of the metadata from DDI-Codebook documents sourced from Dataverse servers.
- Not all tables are populated.
- The ingestion process reads the DDI-C document, applies normalization and matching cleansing steps, and uses a subset of the information to ensure data quality and consistency in the database.

1.2 Surveys, Collections and Distributors

- The `survey` table contains 271 records, including DOI links that connect to the Sciences Po Dataverse server
- The `external_reference` field holds a DOI that resolves to the survey page
- The table holds the study `name` , version date (`date_last_version`), `author` , and `start_date`
- Other fields, all holding the same value, are `language` (always `fr`), ...
- The `subcollection_id` points to one of the 14 entries in the `request_ddi_subcollection` table.
- All sub-collections are associated with two collections in the `request_ddi_collection` table.
- All collections are associated with 'Science Po as the single record in the `request_ddi_distributor` table.

Variables & Questions

- The `representedvariable` table holds a *normalized* version of a question and an optional associated category set.
  - It represents a question more than a variable. It does not hold data type or other variable attributes.
  - RepresentedVariable≡(Normalized Question Text)+(Exact Set of Categories)
  - Represented variables are associated with their categories (if applicable) through the `representedvariable_categories` table.
  - It is independent of the source variable name, type, label, and other attributes.
    - It holds an `internal_label` based on the first variable this question/value information appears in (this does not get updated).
- The `conceptualvariable` table acts as the umbrella that groups all structural variations of the same question text.
  - The 'group' is referenced by the `conceptual_var_id` fields in the represented variable table
- A `is_unique` flag, used in both the represented and conceptual variable tables, indicates whether a variable is a *standard questionnaire item* or an *isolated / non-comparable variable*
  - `false` : The variable has actual verbatim question text. It is treated as semantically meaningful and eligible for longitudinal comparison across survey waves.
  - `true` : The variable has no question text. Because it lacks wording to guarantee semantic equivalence, it is

flagged as is_unique=True.
    - Note that technically, the `is_unique` flag is redundant in the `representedvariable` table, as the same information could be inferred by a null/empty `question_text`. It facilitates querying and, importantly, its value can be manually toggled through the admin UI, even if it has question text (e.g., a one-off experimental question that a curator wants to explicitly exclude from automated longitudinal comparison widgets).
- Note that DDI-C interviewer instructions and pre/post question texts are currently not stored or taken into consideration

Codes Lists

- Harmonized codes and value labels are stored in the `category` table
    - Records in this table are deduplicated based on the code value and normalized category label
    - A `missing` flag is present and populated
- Entries are associated with represented variables through the `representedvariable_categories` table
- Entries are associated with variables through the `bindingvariablecategorystat` table
    - The `stats` column in that table contains the frequency for the associated variable

Concepts

- A `request_ddi_concept` table is present, but no concepts are populated
- The `request_ddi_conceptualvariable_concepts` is likewise empty

1.3 Harmonization and deduplication pipeline

When importing the DDI-Codebook document, the system automatically deduplicates and harmonizes the resources for storage into the various tables.

Code Lists

- Looks up or creates a `category` record matching `code` and `category_label` using a custom case- and accent-insensitive database collation
- The `missing` flag is set when the code/label record is first created
    - ⚠️ The `missing` flag is never subsequently adjusted, even if the same code/category is ingested with a different value, which can technically introduce semantic inconsistencies

Represented & Conceptual Variable Harmonization

To track questions over time and link recurring questions across different survey waves:

- Query by Question Text: Normalizes the question text and searches for existing `RepresentedVariable` entries with the same normalized text.
- Exact Match Check: Compares the set of category IDs of each match.
- If a `RepresentedVariable` with the *same question text* and *identical set of categories* exists, that existing variable is reused.
- Similar Variable (Same Concept): If the question text matches existing variables but the category set differs, a new

`RepresentedVariable` is created and attached to the **existing** `ConceptualVariable`.

- If no matching question text exists, a new `ConceptualVariable` is created, and the new `RepresentedVariable` is linked to it.
- Variables with no categories
- Variables with no question text:
- Variables without a question and code list are ignored

The table below summarizes how the ingestion pipeline treats every permutation of presence or absence of question text and category sets:

Scenario	Question Text?	Categories?	Lookup / Matching Strategy	Harmonization & Database Creation Behavior	`is_unique` Flag	Stats Created?
Standard Categorical Question	✅ Yes	✅ Yes	Matched strictly on normalized `question_text`.	• **Exact category match**: Reuses existing `RepresentedVariable`. • **Category mismatch**: Creates new `RepresentedVariable`, links to **existing** `ConceptualVariable`. • **No question match**: Creates **new** `ConceptualVariable` & `RepresentedVariable`.	`False`	✅ Yes (per category)
Open-ended / Continuous Variable	✅ Yes	❌ No	Matched on normalized `question_text` where categories is empty (`set() == set()`).	• **Match found**: Reuses existing open-ended `RepresentedVariable`. • **Mismatch/New**: Creates new `RepresentedVariable` (and new/reused `ConceptualVariable`).	`False`	❌ No (no categories)
Derived / Calculated Categorical Variable	❌ No	✅ Yes	Fallback matching on `variable_name` across prior `BindingSurveyRepresentedVariable` records.	• **Exact category match**: Reuses existing `RepresentedVariable`. • **Category mismatch**: Creates new `RepresentedVariable`, reuses concept of matching variable name. • **No variable name match**: Creates **new** `ConceptualVariable` & `RepresentedVariable`.	`True`	✅ Yes (per category)
Empty / Unspecified Variable	❌ No	❌ No	None (skipped before lookup).	• **Completely ignored** (`get_or_create_represented_variable` returns `None`). • No database rows or bindings created.	N/A	❌ No

Survey-Variable

- Creates or updates a `BindingSurveyRepresentedVariable` linking the specific `Survey` to the canonical RepresentedVariable`.
- Captures survey-specific properties: `variable_name` (e.g., `ea19_A1`), `universe`, and `notes`.

Category Statistics Binding

- Creates a `BindingVariableCategoryStat` record for each category, associating the frequency count (`stat`) with that specific `BindingSurveyRepresentedVariable`.

---

2. Model Redesign

- Transitioning into a DDI-L and multilingual model strengthens capabilities but naturally increases complexity
- It requires a more sophisticated approach to metadata ingestion and management
- Which features of this enriched model will be leveraged is up to the ReQuest team
    - It largely depends on the needs or requirements of the next version of the application itself, which is beyond the scope of this exercise
    - It can remain simple at first, mimicking the current implementation, and evolve later
    - We assume at this point that minimizing impact on the existing application is a priority
- In the new model, the simple relations between questions, variables, code/categories studies no longer hold true
    - The next iteration of the application will therefore need to take these aspects into account
- This, however, will depend on the content being ingested
    - If the imported DDI is simply based on DDI-C converted to DDI-L, little adjustment will be required
    - Once content is sourced from more complex DDI-L, the application will need to take into account the more versatile resource relationships
        - Questions may exist without associated variables (of any type), studies, or code lists

2.1 Principles

The following principles guided the redesign:

- Inspired by DDI-L, but keep it simple. Only store what is needed by ReQuest. Avoid full DDI-L complexity/coverage.
- Align the table naming conventions on DDI terminology
- Strengthen the model variable cascade (DDI-CDI/GSIM) and other core DDI aspects
- Support for different import/ingest techniques or strategies
- Accommodate current and future versions of DDI-L or similar specifications (DDI-CDI)
- Backward compatibility with DDI-Codebook
- Use JSONB fields to store multilingual strings (faceted strings)
- Rely on URNs as primary identifiers

2.2 Multilingual/Faceted Values

- In addition to strengthening alignment with DDI-L, multilingual support is also a new requirement.
- At the database storage level, this is relatively easy to address by adopting a JSONB storage format for multilingual text values.
- We essentially store it as an array of objects with a language and value property:

```
[
  { "lang": "en", "value": "How old are you?" },
  { "lang": "fr", "value": "Quel âge avez-vous?" }
]
```

- For consistency, the entries must *always* be wrapped in an array, even if we only have a single entry.

```
[{ "lang": "en", "value": "How old are you?" }]
```

- This approach also caters for any additional facets (besides language). This include the ones defined on the DDI-L langString data type (with maxLength, minLength, enumeration, pattern facets) and other ReQuest specific attributes.
- For example, we may want to store a 'normalized' version of a string used internally for comparison or hashing, or keep the 'raw' unsanitized version (which, for example, may contain typos or slight textual variations).

```
[
  { "lang": "en", "value": "how_old_are_you?", "type": "normalized" },
  { "lang": "fr", "value": "Quel âge avez-vous?" },
  { "lang": "fr", "value": "quel_age_avez-vous?", "type": "normalized" },
  { "lang": "fr", "value": "Quellle agge avez vous?   ", "type": "raw" }
]
```

- Other useful facets can include "format" (markdown, HTML), "translated" (to capture automatic or human translations)
- Note that, technically, a string can be stored without a "lang" attribute.

```
[{ "value": "How old are you?" }]
```

- We recommend for this JSONB field type content to be formally defined in a JSON schema and/or Python Pydantic / Django classes.

2.3 Database and resource identifiers

- The new model relies on URNs as primary keys and resource identifiers
  - This requires every resource to have a URN, but this is implied in DDI-L context
  - For resources that come from non-DDI-L sources, such as DDI-C, this would need to be generated (which is relatively easy)
- While auto-generated numeric IDs can still exist in the database, these are not as flexible and reliable:
  - Reimporting the same metadata in different databases (e.g. on different servers) would result in different identifiers. They can therefore not be used as URNs for export purposes
  - Maintaining multiple databases, which is common in an enterprise-grade environment (e.g. staging and production), will translate into inconsistent identifiers, which can complicate scripts or worflows
- In DDI-L:
  - Different identification and referencing mechanisms are available:
    - Identification Sequence: composed of agency id, maintainable id, object id (if not a maintainable), and version
    - URNs: a canonical representation of the identification sequence
  - A legacy deprecated URN format is also documented but should be avoided
  - *All variations must be converted to the canonical URN format*
    - We do not currently store the identification sequence in the tables (only URNs).
  - We leave to the code the responsibility of properly reflecting this in DDI URNs.

- Using standard URNs as primary identifiers also caters for non-DDI resources (e.g. controlled vocabularies)
    - DDI-L resources should carry a valid and properly formatted DDI URNs
    - An ELLST concept is not bound to this requirement
    - ❓ ***Maybe this only applies to specific tables?***
- ❓ ***Do we want the database to enforce proper URN formatting (table constraint or code level check)***

2.4 Hashing

- As we transition into a more complex and felxible model, we anticpate the need for different comparison, normalization, or hamonization hashing techniques to be used (for processing or publication purposes)
- We support this be allowing mullitple hashes or signatures to be stored on a resource
- These could be computed using different algorithms or set of attributes
- The ReQuest applications or indexer (elastic search) can slectively choose which to use

Example:

```
[
{"normalized-fr":"8d734..."}
{"normalized-en":"7ff4s5..."}
{"normalized-i18n":af76cc76...""}
{"request-v1":"8d09ff..."}
{"request-v2":"63ab009..."}
{"request-v2-codes":"6526fbc..."}
{"request-v2-categories":"e65d78..."}
{"ai-agent-0001":6fb8s7...","llm":"gemini-flash-3.7"}
]
```

2.5 Extended Attributes

- Modeling all the attributes and characteristics of DDI-L resources in a database would be unrealistic. SQL is not the right technology for this.
- And the ReQuest application only uses a handful of them for comparison or information purposes.
- Capturing a small subset of these elements can however be desirable for internal or future application needs. For example:
    - A question's intent, instructions, or instrument
    - A category definition, inclusion/exclusion
- This can also vary depending on the metadata provider, the incoming DDI content, or the DDI profile
- To support this generically, we use an `extended_attributes` JSON field wherever applicable
- We recommend embedding a schema definition or URL in the stored JSON (Self-Describing JSON Document) to identify its content
    - The JSON could be validated against the schema (inside or outside the database)

Example: a question coming from DDI-L carrying few extra properties of interest (the question text is already in the table `question_text` field):

```
[
{
  "$schema": "https://request.sciencespo.fr/schemas/ddil-question-item.json",
  "urn": "urn:ddi:int.example:c7233173-2d8f-4a3f-918b-60ee08d61d5d:1",
  "Description": "This is a great question",
  "QuestionIntent": "Get a great answer"
  ]
}
{
  "$schema": "https://request.sciencespo.fr/schemas/ddil-question-item.json",
  "urn": "urn:ddi:int.example:c7233173-2d8f-4a3f-918b-60ee08d61d5d:1",
  "Description": "This is a great question",
  "QuestionIntent": "Get a great answer"
  ]
}
]
```

Example: a question coming from DDI-Codebook with surrounding elements (the question text is already in the table's' `question_text` field):

```
[
{
  "$schema": "https://request.sciencespo.fr/schemas/ddic-qstn.json",
  "urn": "urn:some:var:ddiCodebookUrn",
  "preQTxt": "I will now ask you a great question",
  "postQTxt": "Thank you for your great answer",
  "ivuInstr": "Be engaged and excited while asking this great question"
  ]
}
]
```

2.6 ReQuest Resource Association Paths

- In DDI, relationships between resource can involve complex, multi-hop paths
  - See for example variable_question_relationships.md
- This level of detail is not needed for ReQuest, and we insted use 'shortcuts' to capture associations that are relevant for the application
- Tables have been modelled to relate the following:
  - QuestionItem with InstanceVariable
  - Instrument with QuestionItem
  - StudyUnit with InstanceVariable
- These tables hold the source and target URNs along with a 'path' expression to document the referencing chain as

needed
-

2.7 Semantic Relationships

- In the context of ReQuest, we may need to capture relationships that are more *semantic* in nature, not supported by DDI-L (e.g. using SKOS, XKOS, PROV-O, and the like)
- A generic table has been included in the model to support this
- This is a minimalistic RDF triple-store
- Fields include
  - `subject_type` and `subject_urn` to represent the source
  - `predicate` : to describe the relationship
  - `object_type` and `object_urn` to represent the target
- The typing fields are needed, as URNs do not inform on the resource type
- Note that there are no constraints on the content of these fields. It is up to the implementation to manage the

Some examples:

Predicate	Source Type	Target Type	Example Scenario
skos:exactMatch	`QuestionItem`	`QuestionItem`	Bilingual questionnaire translation equivalence.
skos:closeMatch	`QuestionItem`	`QuestionItem`	Aligning minor wording changes across survey waves.
skos:broadMatch	`Category`	`Category`	Collapsing a detailed 4-digit ISCO job code into a 1-digit major group.
prov:wasDerivedFrom	`HarmonizedVariable`	`SourceVariable`	Tracking source variables rolled into a synthetic composite index.
prov:wasDerivedFrom	`HarmonizedQuestion`	`SourceQuestion`	Synthesizing a single canonical national question from three regional variants.
ddi:measures	`QuestionItem`	`ConceptualVariable`	Linking an elicitation prompt to the abstract concept it quantifies.

2.8 JSONB Fields

- To simplify the model and for flexibility, we use JSONB fields holding URN references to various DDI resources
- One drawback if that referential integrity cannot be enforced directly by Postgres.
  - This should only affect resource deletion (as URNs should not change after a resource has been created)
  - Various mechanisms can be implemented to address this, including triggers, application code, and garbage collection
- Also note that, when a JSON or JSONB document exceeds this 2 KB limit, Postgres automatically triggers a two-step "offload" process

    - Compression: Postgres first attempts to compress the JSON string using pglz or lz4 to force it back under the 2 KB limit.
    - Out-of-line Offloading: If the compressed document is still larger than 2 KB, it is sliced into chunks and moved completely out of the main table into a hidden companion table called a TOAST table. The main table row preserves only an 18-byte pointer to those chunks.
- While this is unlikely to be noticeable in the context of ReQuest performance, it is important to keep in mind (don't store very large JSON objects)

---

3. Database Model

See the database.md document for detailed information on the database model and table definitions.

3.1 Summary of Changes

- Questions are now in a dedicated `QuestionItem` table
    - A Question can be directly associated with response domain (e.g. CodeList)
- All three variable cascade entities are now explicitly in the model:
    - `RepresentedVariable` can still be used as a bridge between questions and value representation (e.g., code list)
    - `InstanceVariable` essentially replaces the binding survey variables table
- New `<type>Scheme` tables have been added to support maintenance containers for relevant resources (concepts, questions, variables, categories, codes)
- Collection and sub-collections have been replaced by the more generic `Group` resource (can be nested and used for grouping various resources, including studies)
- Distributor has been replaced by `Organization` (distributor being a type of)
- An `Instrument` table has been introduced for questions related to questionnaires (instead of or in addition to study)
- Various utility tables have been introduced to:
    - Describe simplified ReQuest resource relationships (question, variables, studies) and alleviate the need for storing complex DDI paths
    - Centrally register URNs for referential integrity, polymorphic type resolution (URN/type resolution), and other purposes
    - Capture optional "semantic" relationships between resources that are not in DDI (e.g. closeMatch, exactMatch, basedOn)
    - Capture historical events for resources (log)
    - Support phase 2 import and ingestion

3.2 Common DDI resource fields

The tables representing DDI resources typically contain the following fields:

- `urn` : the primary key and identifier. Typically a well-formed DDI-L URN, but not always the case (e.g., ELLST concept)

    - Django models (DDIIdentifiable) and Pydantic schemas (DDIIdentifiableSchema) provide property helpers to parse and decompose URNs.
- `name`, `label`, `description`: commonly used on DDI resources
- `extended_attributes`: to store resource attributes that are not present in the fields.
- `hashes`: a JSON field to hold a list of hashes computed for this resource. The object should at least identify the algorithm and the computed value. Additional attributes can be added as needed.

3.3 QuestionItem / QuestionScheme

- As this is the core focus of Request, this is the heart of the model
- The `QuestionItem` table stores the normalize/harmonized ReQuest question
    - This was previously stored in a RepresentedVariable, but has now been decoupled
    - It can have be used by a RepresentedVariable, but does not require one
    - It can also be used by more than one RepresentedVariable (and other resources)
- A QuestionItem can have ResponseDomain, which can be a CodeDomain, and therefore be directly associated with a CodeList
    - As a consequence, a variable is not necessary to associate a question with a code list
    - We currently have both a `code_list_urn` and a generic `response_domain` JSONB object that can carry the many options supported by DDI-L
    - If the question has a CodeDomain, the JSON will hold a URN pointing to the CodeList in the database
    - ❓ ***Do you want the flexibility of response domain, or do we only need support for code list? Or both?***
    - ❓ ***A QuestionItem can also be associated with a CategoryDomain. Is this a use case we want to support?***
- It can be related to:
    - An Instrument through the `InstrumentQuestion` table
    - A RepresentedVariable through the `QuestionVariable` table
        - It is important to note that in DDI-L, the Question are related to Variable (not RepresentedVariable)
        - This is a difference with the GSIM/DDI-CDI variable cascade (see GSIM/CDI vs DDI-L Cascade)
        - Alignment on GSIM/DDI-CDI is a recommended option
- We have elected not to model QuestionGroup in the database. Generic Group can be used for ReQuest purposes.
- We have elected at this time not to model QuestionGrid. Grouping mechanims could be in the meantime.

3.4 Codes / Categories

- Unliked in the current implementation of ReQuest, codes and categories in DDI-L are separate resources
- A `Category` is essentially the use of a concept for the purpose of classification or categorization
- Categories are defined in a `CategoryScheme` table
- Codes live in a `CodeList` and are not reusable. They reference a Category.
- Both codes and categories carry a `is_missing` attribute.
- Hierarchical codes or categories are supported through a `parent_urn` property (DDI-L implements the other way

around)

- We have elected not to model CategoryGroup in the database. Generic Group can be used for ReQuest purposes.

3.5 Variables

- The three types of resources composing the variable cascade are represented through two tables:
  - `ConceptualVariable` and `ConceptualVariableScheme`
  - `RepresentedVariable` and `RepresentedVariableScheme`
    - Note that it is based on the variable cascade as defined in GSIM and DDI-CDI (see GSIM/CDI vs DDI-L Cascade)
  - `InstanceVariable` and `InstanceVariableScheme` (this technically replaces the)
- These variables only carry a small subset of attributes, so they are, at this time, very similar tables. This could be extended.
  - Conceptual variable: can have a concept but has no Unit Type (e.g., Person)
  - Represented variable: only representation used in ReQuest is "code" (as a reference to code list). But this is a narrow view. The Universe is also not taken into consideration (e.g., Students)
  - Instance Variable: just a resource to connect to a StudyUnit (technically through a dataset). The population (e.g., Students in School District A in 2019) and other attributes are not documented (but likely not relevant for ReQuest)
- We have elected at this time not to model Unit, Universe, and Population and related schemes associated with the variable cascade
  - A `universe` property is present in the current ReQuestdatabase, but is a text field in the `BindingSurveyRepresentedVariable` table, which is essentially an InstanceVarible
  - This is currently populated from DDI-C var/universe property (which would lbe the equiavelnt of Population in the variable cascade)
  - We will for now store this in the DatafileVariable extended attributes
- For RepresentedVariable
  - We currently have both a `code_list_urn` and a generic `value_representation` JSONB object that can carry the many options supported by DDI-L
    - If the variable has a CodeRepresentation, the JSON will hold a URN pointing to the CodeList in the database
  - ❓ ***Do you want the flexibility of value representation, or do we only need support for code list? Or both?***

3.6 StudyUnit

- A StudyUnit table is used at this time to represent studies
- Unllike in the current version of request, a Datafile exists to capture the relationship between studies and their instance variables
  - This is the natural relationship betweem variable and studies modelled by all DDI specifications
  - The term was use to reflect the the relationship with an actual physical file nstance
  - ThIt is also necessary to be able to store frequencies and statistics

- The DatafileVariable table links Instancevariables to Dataset
  - A `statistics` JSONB fields can be used to store optional summary and descriptiove statistics
- The DatafileFrequency table can be used to store optional frequencies

3.7 Group / GroupItem Tables

- `Group` reflects the generic mechanism for bringing together DDI resources for non-administrative purposes (vs specialized grouping under a scheme)
- A DDI-L Group can hold references to many different types of resources
- The `GroupItem` table is used to enumerate the members (which can be a subgroup for nesting)
- The primary purpose if to replace the current collection and subcollection tables for the study units to organization relationships

3.8 Organization

- Replaces the legacy generic `Distributor` table with a standard DDI-L `Organization` resource.
- Inherits canonical URN primary key (`urn`) and multi-algorithm hash digests (`hashes`) via `DDIIdentifiable` .
- Features multilingual organization name (`name`), multilingual acronyms/nicknames (`nickname`), mission/descriptions (`description`), authoritative URI (`uri`), institutional email (`email`), phone (`phone`), organization classification (`organization_type`), parent-child hierarchical nesting (`parent_organization_urn`), and extensible attribute storage (`extended_attributes`).
- Serves as the top-level institutional anchor for collections and study series (`Collection.organization_urn`).

3.9 ReQuest Relationships Tables

- As mentioned, there are many different ways in DDI-L to relate different resources with each other, often involving multiple references
- Modeling all these possibilities would bloat the database with many tables. SQL is not the right technology for such a purpose
- We instead use shortcut tables to keep the code simple. Th includes `StudyInstrument` , `InstrumentQuestion` , `QuestionVariable` and `DatafileVariable`

3.10 Support Tables

- The `UrnRegistry` table provides a unique index for resource URNs and their types/class. Extended attributes can be used for any practical purposes (e.g., controlling visibility, indexing status, internal attributes).
- The `SemanticRelationship` table is available to capture non-DDI relantionship between resources
- An `EventLog` table is available to keep track of the history of events, actions, or notes on the various resources
- Ingestion tables are work in progress
  - The `StageImport` , `StageResourceNode` , `UrnAlias` , and `MetadataQuarantine` tables are in the model bt will be further explored/discussed during Phase-2
  - We can technically directly ingest DDI-L into the above model, but may want the 'raw' metadata loaded in the database for practical reasons (isolation, quality assurance, quarantine, and reference)

4. Conclusions and Next Steps

- This initial version of the model and database provides a significant step up and benefits over the current model
- It can cater to both the current and future needs of ReQuesr
- It will likely evolve as we progress through phase II, as we put it to practical use
- Next steps will focus on:
    - MVP Import/ingestion packages using selected/focused DDI-L use cases (to be identified)
    - Export package
    - Support for DDI-Codebook